1

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>

void merge(int arr[], int l, int m, int r) {
    int i = l, j = m + 1, k = 0;
    int temp[r - l + 1];

    while (i <= m && j <= r) {
        if (arr[i] <= arr[j])
            temp[k++] = arr[i++];
        else
            temp[k++] = arr[j++];
    }
    while (i <= m) temp[k++] = arr[i++];
    while (j <= r) temp[k++] = arr[j++];

    for (i = l, k = 0; i <= r; i++, k++)
        arr[i] = temp[k];
}

void sequentialMergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;
        sequentialMergeSort(arr, l, m);
        sequentialMergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

void parallelMergeSort(int arr[], int l, int r) {
    if (l < r) {
        int m = (l + r) / 2;

        #pragma omp parallel sections
        {
            #pragma omp section
            parallelMergeSort(arr, l, m);

            #pragma omp section
            parallelMergeSort(arr, m + 1, r);
        }
```

```
        merge(arr, l, m, r);
    }
}

int main() {
    int n = 100000;
    int arr1[n], arr2[n];

    for (int i = 0; i < n; i++) {
        arr1[i] = rand() % 1000;
        arr2[i] = arr1[i];
    }

    double start, end;

    start = omp_get_wtime();
    sequentialMergeSort(arr1, 0, n - 1);
    end = omp_get_wtime();
    printf("Sequential Merge Sort Time: %f seconds\n", end - start);

    start = omp_get_wtime();
    parallelMergeSort(arr2, 0, n - 1);
    end = omp_get_wtime();
    printf("Parallel Merge Sort Time: %f seconds\n", end - start);

    return 0;
}
```

2

```
#include <stdio.h>
#include <omp.h>

int main() {
    int n;
    printf("Enter number of iterations: ");
    scanf("%d", &n);

    int thread_start[100], thread_end[100];
    int i;

    for (i = 0; i < 100; i++) {
        thread_start[i] = -1;
        thread_end[i] = -1;
```

```
    }

    #pragma omp parallel for schedule(static,2)
    for (i = 0; i < n; i++) {
        int tid = omp_get_thread_num();

        if (thread_start[tid] == -1)
            thread_start[tid] = i;
        thread_end[tid] = i;
    }

    for (i = 0; i < 100; i++) {
        if (thread_start[i] != -1) {
            printf("Thread %d : Iterations %d -- %d\n", i, thread_start[i], thread_end[i]);

        }
    }

    return 0;
}
```

3

```
#include <stdio.h>
#include <omp.h>

int fib(int n) {
    int x, y;

    if (n < 2)
        return n;

    #pragma omp task shared(x)
    x = fib(n - 1);

    #pragma omp task shared(y)
    y = fib(n - 2);

    #pragma omp taskwait
    return x + y;
}

int main() {
```

```
    int n;

    printf("Enter number of Fibonacci terms: ");
    scanf("%d", &n);

    printf("Fibonacci Series:\n");

    for (int i = 0; i < n; i++) {
        int result;

        #pragma omp parallel
        {
            #pragma omp single
            {
                result = fib(i);
            }
        }

        printf("%d ", result);
    }

    printf("\n");
    return 0;
}
```

4

```
#include <stdio.h>
#include <math.h>
#include <omp.h>

int is_prime(int num) {
    if (num < 2) return 0;
    for (int i = 2; i <= sqrt(num); i++) {
        if (num % i == 0)
            return 0;
    }
    return 1;
}

int main() {
    int n;
    printf("Enter the value of n: ");
    scanf("%d", &n);
```

```
    printf("\nPrime numbers from 1 to %d:\n", n);
    for (int i = 1; i <= n; i++) {
        if (is_prime(i))
            printf("%d ", i);
    }
    printf("\n");

    double start_time, end_time;

    start_time = omp_get_wtime();
    for (int i = 1; i <= n; i++) {
        is_prime(i);
    }
    end_time = omp_get_wtime();
    printf("Serial Time: %f seconds\n", end_time - start_time);

    start_time = omp_get_wtime();
    #pragma omp parallel for
    for (int i = 1; i <= n; i++) {
        is_prime(i);
    }
    end_time = omp_get_wtime();
    printf("Parallel Time: %f seconds\n", end_time - start_time);

    return 0;
}
```

5

```
nano sr.c
paste this
#include <mpi.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char** argv) {
    int rank, size;
    MPI_Status status;

    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get current process rank and total number of processes
```

```
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Process 0 sends a message to all other processes
    if (rank == 0) {
        for (int i = 1; i < size; i++) {
            char message[100];
            sprintf(message, "Hello from process 0 to process %d", i);
            MPI_Send(message, strlen(message) + 1, MPI_CHAR, i, 0, MPI_COMM_WORLD);
            printf("Process 0 sent message to process %d\n", i);
        }
    } else {
        // All other processes receive the message from process 0
        char recv_buf[100];
        MPI_Recv(recv_buf, 100, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &status);
        printf("Process %d received message: %s\n", rank, recv_buf);
    }

    // Finalize MPI
    MPI_Finalize();
    return 0;
}

ctrl+o>enter>ctrl+x
mpicc sr.c -o sr
mpirun –oversubscribe --np 6 ./sr



6
nano deadlock.c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, send_data, recv_data;

    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    send_data = (rank == 0) ? 100 : 200;

    if (rank == 0) {
        printf("Process 0: Before Send\n");
```

```
        fflush(stdout);

        MPI_Send(&send_data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);  // blocking send

        printf("Process 0: After Send, Before Recv\n");
        fflush(stdout);

        MPI_Recv(&recv_data, 1, MPI_INT, 1, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        printf("Process 0: Received %d\n", recv_data);
        fflush(stdout);
    } else if (rank == 1) {
        printf("Process 1: Before Send\n");
        fflush(stdout);

        MPI_Send(&send_data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);  // blocking send

        printf("Process 1: After Send, Before Recv\n");
        fflush(stdout);

        MPI_Recv(&recv_data, 1, MPI_INT, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);

        printf("Process 1: Received %d\n", recv_data);
        fflush(stdout);
    }

    MPI_Finalize();
    return 0;
}

ctrl+o>enter>ctrl+x
mpicc deadlock.c -o deadlock
mpirun --np 2 ./deadlock


7
nano  broadcast.c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size;
    int number;
```

```
    // Initialize MPI environment
    MPI_Init(&argc, &argv);

    // Get current process rank and total number of processes
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (rank == 0) {
        number = 42;  // The value to be broadcasted
        printf("Process 0 is broadcasting number %d\n", number);
    }

    // Broadcast the value of `number` from process 0 to all others
    MPI_Bcast(&number, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // All processes print the received number
    printf("Process %d received number %d\n", rank, number);

    // Finalize MPI
    MPI_Finalize();
    return 0;
}

ctrl+o>enter>ctrl+x
mpicc broadcast.c -o broadcast
mpirun –oversubscribe --np 4 ./broadcast

8
nano sg.c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size;
    int data[100];         // Only used by root
    int recv_value;        // Value received by each process
    int gathered[100];     // Only used by root

    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get rank and size
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
```

```
    // Only rank 0 initializes data
    if (rank == 0) {
        for (int i = 0; i < size; i++) {
            data[i] = i * 10;
        }
        printf("Process 0 initialized data: ");
        for (int i = 0; i < size; i++) {
            printf("%d ", data[i]);
        }
        printf("\n");
    }

    // SCATTER: One value from `data` array is sent to each process
    MPI_Scatter(data, 1, MPI_INT, &recv_value, 1, MPI_INT, 0, MPI_COMM_WORLD);
    printf("Process %d received value %d from Scatter\n", rank, recv_value);

    // Each process modifies its value
    recv_value += rank;

    // GATHER: Each process sends modified value back to root
    MPI_Gather(&recv_value, 1, MPI_INT, gathered, 1, MPI_INT, 0, MPI_COMM_WORLD);

    // Only rank 0 displays gathered results
    if (rank == 0) {
        printf("Process 0 gathered data: ");
        for (int i = 0; i < size; i++) {
            printf("%d ", gathered[i]);
        }
        printf("\n");
    }

    // Finalize
    MPI_Finalize();
    return 0;
}

ctrl+o>enter>ctrl+x
mpicc sg.c -o sg
mpirun –oversubscribe --np 4 ./sg
```

```
nano pg9.c
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size;
    int value;
    int sum_result, prod_result, max_result, min_result;
    int all_sum, all_prod, all_max, all_min;

    // Initialize MPI
    MPI_Init(&argc, &argv);

    // Get rank and size
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Each process sets its value
    value = rank + 1;  // Example: rank 0 = 1, rank 1 = 2, etc.
    printf("Process %d has value %d\n", rank, value);

    // ---------- MPI_Reduce (result only at root, rank 0) ----------
    MPI_Reduce(&value, &sum_result, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD);
    MPI_Reduce(&value, &prod_result, 1, MPI_INT, MPI_PROD, 1, MPI_COMM_WORLD);
    MPI_Reduce(&value, &max_result, 1, MPI_INT, MPI_MAX, 0, MPI_COMM_WORLD);
    MPI_Reduce(&value, &min_result, 1, MPI_INT, MPI_MIN, 0, MPI_COMM_WORLD);

    if (rank == 0) {
        printf("\n--- Results using MPI_Reduce (only on root) ---\n");
        printf("Sum   = %d\n", sum_result);
        printf("Prod  = %d\n", prod_result);
        printf("Max   = %d\n", max_result);
        printf("Min   = %d\n", min_result);
    }

    // ---------- MPI_Allreduce (result sent to all processes) ----------
    MPI_Allreduce(&value, &all_sum, 1, MPI_INT, MPI_SUM, MPI_COMM_WORLD);
    MPI_Allreduce(&value, &all_prod, 1, MPI_INT, MPI_PROD, MPI_COMM_WORLD);
    MPI_Allreduce(&value, &all_max, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);
    MPI_Allreduce(&value, &all_min, 1, MPI_INT, MPI_MIN, MPI_COMM_WORLD);

    printf("Process %d - AllReduce: Sum=%d Prod=%d Max=%d Min=%d\n",
           rank, all_sum, all_prod, all_max, all_min);
```

```
    // Finalize
    MPI_Finalize();
    return 0;
}
```

ctrl+o>enter>ctrl+x
mpicc pg9.c -o pg9
mpirun --np 2 ./pg9